

Chapter 7: Pretrained Models for Machine Translation

Machine Translation: Advanced Topics

Vincent Vandeghinste

Why pretrained models for MT?

Training NMT systems from scratch requires:

- large parallel corpora;
- substantial computational resources;
- enough domain-specific examples;
- enough data for each language pair.

This is difficult for:

- low-resource languages;
- specialised domains;
- non-English-centric translation directions.

The core idea

Pretraining separates learning into two stages:

1. Pretraining

The model is trained on very large amounts of data to acquire general linguistic, multilingual or translational knowledge.

2. Fine-tuning

The pretrained model is adapted to a specific task, language pair or domain using supervised data.

This is a form of **transfer learning**.

Transfer learning

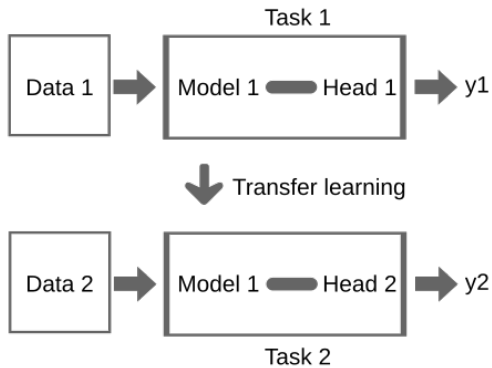


Image source for full notes: Wikipedia, Transfer learning.

Pretraining versus training from scratch

	Training from scratch	Pretraining + fine-tuning
Initial parameters	Random	Already trained
Data needed	Usually large parallel corpus	Less task-specific data needed
Learning focus	Everything at once	Adapt existing representations
Low-resource MT	Difficult	Often more effective
Reuse	Limited	High

What does pretraining learn?

During pretraining, the model may learn:

- lexical and subword representations;
- syntactic and semantic regularities;
- long-range dependencies;
- discourse patterns;
- cross-lingual correspondences;
- fluent sequence generation.

Fine-tuning can then focus more on **source–target alignment**.

Main pretraining strategies

Most pretrained MT models fall into three broad paradigms:

- 1 **Denoising autoencoding**
reconstruct clean text from corrupted text.
- 2 **Multilingual denoising**
apply denoising across many languages with shared parameters.
- 3 **Massively multilingual supervised translation**
train directly on large amounts of parallel data.

Denoising autoencoding

Original sentence

She eats apples every day .

Corrupted input

She <mask> apples day .

Target output

She eats apples every day .

No translation is involved, but the model learns to map an imperfect input sequence to a well-formed output sequence.

Multilingual denoising

The same denoising objective can be applied across languages.

English

She <mask> apples day .

Target: She eats apples every day .

Dutch

Zij <mask> dag appels .

Target: Zij eet elke dag appels .

Shared parameters encourage cross-lingual transfer.

From multilingual NMT to pretrained MT

- Earlier multilingual NMT systems were trained directly on parallel data for several language pairs.
- Shared vocabularies and language tags already created transfer effects.
- Later models made pretraining an explicit design choice.
- Some models learn from monolingual denoising first.
- Others are trained directly as massive reusable translation systems.

Period	Development
Early NMT	Encoder–decoder models trained from scratch on parallel data
2017	Transformer architecture enables large-scale training
2018–2019	ELMo/BERT popularise self-supervised pretraining
2019–2021	T5, mT5 and mBART bring pretraining to encoder–decoder models
2021–2023	M2M-100, NLLB and MADLAD scale multilingual MT

Models covered in this chapter

- 1 T5: Text-to-Text Transfer Transformer
- 2 mT5: multilingual T5
- 3 mBART: multilingual denoising for seq2seq
- 4 M2M-100: many-to-many supervised MT
- 5 NLLB: massively multilingual low-resource MT
- 6 MADLAD-400: multilingual text-to-text translation

A consistent comparison structure

For each model, we ask the same questions:

- What is the core idea?
- What architecture does it use?
- What training objective does it use?
- How is translation specified?
- What are the strengths and limitations?
- How do we use it in the hands-on session?

T5: core idea

T5 stands for **Text-to-Text Transfer Transformer**.

Its main idea:

Every NLP task is text-to-text generation

Input text contains the task specification. Output text contains the answer.

Examples:

- summarisation: input text $\rightarrow$ summary;
- classification: input text $\rightarrow$ label;
- translation: source sentence $\rightarrow$ target sentence.

T5: architecture and objective

- Standard Transformer encoder–decoder architecture.
- Encoder reads the input sequence.
- Decoder generates the output sequence autoregressively.
- Pretrained with **span corruption**.

Span corruption example

Input: Thank you <X> me to your party <Y> week.

Target: <X> for inviting <Y> last <Z>

T5: translation control

T5 uses a natural-language prefix.

Fine-tuning example

Input: translate English to Dutch: The cat sits on the mat.

Target: De kat zit op de mat.

The prefix tells the model which task to perform.

T5: strengths and limitations

Strengths

- simple text-to-text interface;
- reusable across many NLP tasks;
- suitable architecture for MT;
- pretraining improves starting point.

Limitations

- pretraining is not translation-specific;
- not inherently multilingual in original T5;
- supervised fine-tuning remains essential.

Kaggle session

[Open T5 Notebook](#)

- Fine-tune pretrained T5 on EN-NL
- Observe training vs validation loss
- Evaluate with BLEU and chrF

T5 hands-on: training

```
!python finetune_pretrained_t5.py \  
  --src-file /kaggle/input/datasets/vincentvandeghinste/tatoeba-en-nl/train.en \  
  --tgt-file /kaggle/input/datasets/vincentvandeghinste/tatoeba-en-nl/train.nl \  
  --src-val /kaggle/input/datasets/vincentvandeghinste/tatoeba-en-nl/dev.en \  
  --tgt-val /kaggle/input/datasets/vincentvandeghinste/tatoeba-en-nl/dev.nl \  
  --epochs 10 \  
  --batch-size 16 \  
  --lr 5e-5 \  
  --save t5_en_nl \  
  --eval-metrics \  
  --history-json t5.hist \  
  --show-val-examples 5 \  
  --max-src-len 128 --max-tgt-len 128 \  
  --num-beams 4 --max-gen-len 128
```

T5: training results

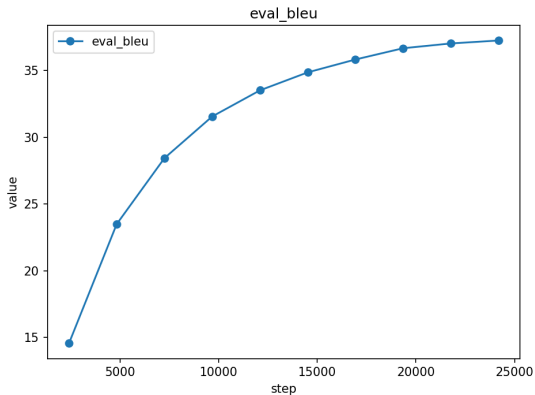
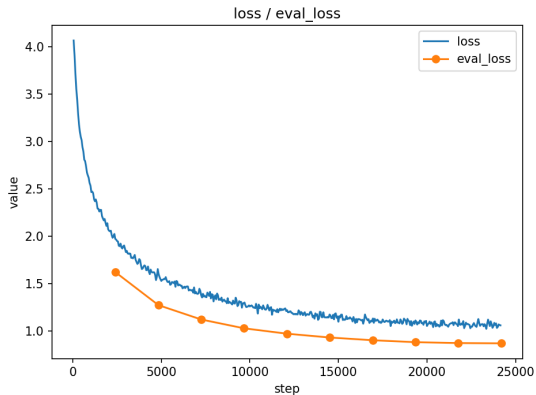


Figure: T5 fine-tuning: loss and BLEU over epochs

T5 hands-on: translation and evaluation

```
!python pretrained_translate.py \  
  --model-dir t5_en_nl \  
  --src-file /kaggle/input/datasets/vincentvandeghinste/tatoeba-en-nl/test.en \  
  --out-file test.t5.nl \  
  --ref-file /kaggle/input/datasets/vincentvandeghinste/tatoeba-en-nl/test.nl \  
  --metrics bleu,chrp \  
  --batch-size 32 \  
  --max-src-len 128 \  
  --max-gen-len 128 \  
  --num-beams 4
```

T5: interpreting results

- Evaluation loss can start much lower than models trained from scratch.
- Pretraining provides useful linguistic knowledge.
- BLEU may still remain lower than a Transformer trained from scratch on this dataset.
- A small parallel corpus may be insufficient to fully adapt the model.

mT5 extends T5 to the multilingual setting.

It keeps the same basic design:

- encoder–decoder Transformer;
- text-to-text formulation;
- span-corruption objective.

The key difference is multilingual pretraining on **mC4** (multilingual Colossal Clean Crawled Corpus), covering more than 100 languages.

	T5	mT5
Architecture	Encoder–decoder	Encoder–decoder
Objective	Span corruption	Span corruption
Data	Mainly English	Multilingual mC4
Tokenizer	English-oriented	Multilingual
Translation control	Text prefix	Text prefix

From the user perspective, mT5 looks similar to T5.

Example

Input: translate English to Dutch: The cat sits on the mat.

Target: De kat zit op de mat.

The difference is that the model has already seen many languages during pretraining.

mT5: strengths and limitations

Strengths

- multilingual pretraining;
- better non-English coverage than T5;
- multilingual tokenizer;
- useful for multilingual transfer.

Limitations

- still denoising-based;
- not explicitly trained as an MT model;
- requires supervised fine-tuning;
- can be memory-intensive.

Kaggle session

[Open mT5 Notebook](#)

- Multilingual version of T5
- Compare with monolingual T5
- Observe effect of smaller batch size

mT5 hands-on: training

```
!python finetune_pretrained_t5.py \  
  --pretrained-model google/mt5-small \  
  --src-file /kaggle/input/datasets/vincentvandeghinste/tatoeba-en-nl/train.en \  
  --tgt-file /kaggle/input/datasets/vincentvandeghinste/tatoeba-en-nl/train.nl \  
  --src-val /kaggle/input/datasets/vincentvandeghinste/tatoeba-en-nl/dev.en \  
  --tgt-val /kaggle/input/datasets/vincentvandeghinste/tatoeba-en-nl/dev.nl \  
  --epochs 10 \  
  --batch-size 8 \  
  --lr 5e-5 \  
  --save mt5_en_nl \  
  --eval-metrics \  
  --history-json mt5.hist \  
  --show-val-examples 5
```

mT5: training results

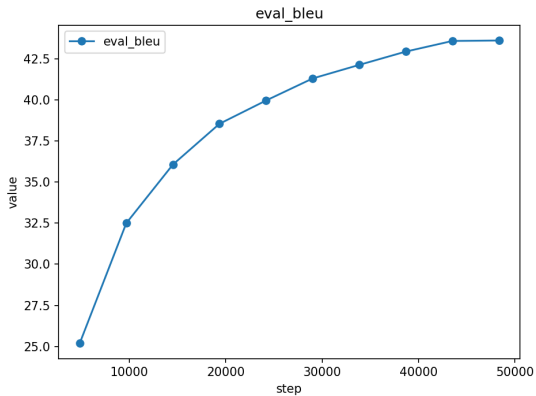
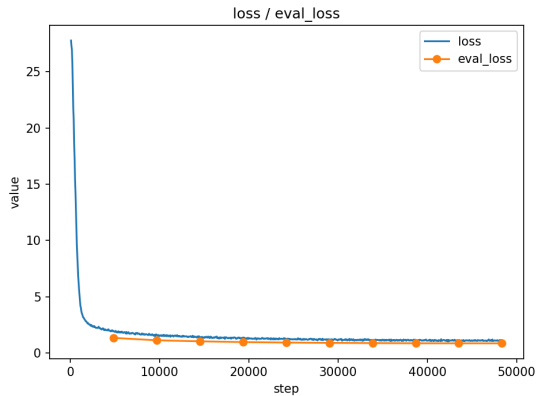


Figure: mT5 fine-tuning: loss and BLEU over epochs

mT5: interpreting results

- Training and evaluation loss can converge quickly.
- BLEU may continue improving after the loss appears stable.
- Smaller batch size increases the number of optimisation steps.
- Multilingual pretraining helps, but task-specific fine-tuning still matters.

mBART extends BART to multilingual sequence-to-sequence learning.

It was designed more explicitly for multilingual seq2seq tasks such as translation.

Main idea

Pretrain one encoder-decoder model with multilingual denoising, then fine-tune it for MT.

mBART: denoising objective

- Pretrained on monolingual documents from multiple languages.
- Input text is corrupted.
- The model reconstructs the original text.

Example

Original: The cat sits on the mat.

Corrupted input: The <mask> on the mat sits cat.

Target: The cat sits on the mat.

Unlike T5 and mT5, mBART uses explicit language tags.

Translation example

Input: <en_XX> The cat sits on the mat.

Target: <nl_XX> De kat zit op de mat.

The source tag identifies the input language. The target tag guides the decoder.

mBART language codes

Code	Language
en_XX	English, generic variant
nl_XX	Dutch, generic variant
fr_XX	French, generic variant
de_DE	German, Germany
es_XX	Spanish, generic variant

The first part identifies the language. The second part identifies a region or generic variant.

Full list

[View full language list \(HuggingFace\)](#)

Why the exact language code matters

Language codes are not arbitrary labels.

They are:

- part of the tokenizer vocabulary;
- used during pretraining;
- expected exactly by the model.

Wrong codes can lead to:

- wrong target language;
- poor translations;
- unstable generation.

mBART: strengths and limitations

Strengths

- explicitly multilingual seq2seq model;
- strong transfer to MT;
- useful for low-resource settings;
- language tags provide control.

Limitations

- pretraining is still denoising, not translation;
- exact language codes are required;
- can be memory-intensive.

Kaggle session

[Open mBART Notebook](#)

- Use language tags (en_XX $\rightarrow$ nl_XX)
- Fine-tune multilingual denoising model
- Compare behaviour with T5/mT5

mBART hands-on: training

```
!python finetune_pretrained_mbart.py \  
  --src-file /kaggle/input/datasets/vincentvandeghinste/tatoeba-en-nl/train.en \  
  --tgt-file /kaggle/input/datasets/vincentvandeghinste/tatoeba-en-nl/train.nl \  
  --src-val /kaggle/input/datasets/vincentvandeghinste/tatoeba-en-nl/dev.en \  
  --tgt-val /kaggle/input/datasets/vincentvandeghinste/tatoeba-en-nl/dev.nl \  
  --src-lang en_XX \  
  --tgt-lang nl_XX \  
  --epochs 10 \  
  --batch-size 8 \  
  --lr 5e-5 \  
  --save mbart_en_nl \  
  --eval-metrics \  
  --history-json mbart.hist \  
  --show-val-examples 5
```

mBART: training results

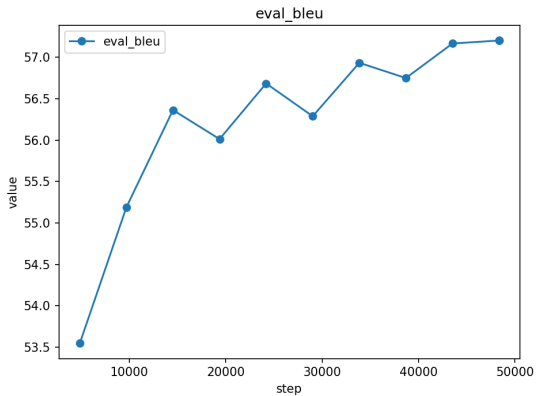
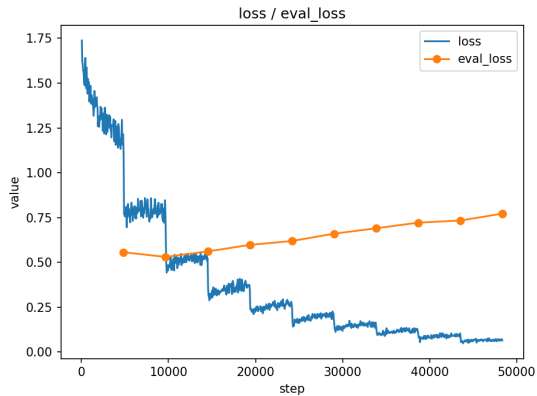


Figure: mBART fine-tuning: loss and BLEU over epochs

- Training loss may improve more clearly than development loss.
- BLEU can still increase over epochs.
- Translation quality may keep improving even when loss curves are hard to interpret.
- This shows why MT-specific evaluation remains important.

M2M-100 was developed for direct many-to-many translation.

Goal

Translate between many language pairs without always pivoting through English.

Unlike T5, mT5 and mBART, M2M-100 is trained directly on large-scale parallel translation data.

- Standard Transformer encoder–decoder model.
- Trained on multilingual parallel corpora.
- Shared parameters across many translation directions.
- Learns translation directly during large-scale supervised training.

This makes it a reusable pretrained translation model rather than a general denoising model.

M2M-100: language codes

M2M-100 uses simple language identifiers.

Code	Language
en	English
nl	Dutch
fr	French
de	German
es	Spanish

Example arguments: `-src-lang en` and `-tgt-lang nl`.

Full list

[View all languages \(HuggingFace\)](#)

M2M-100: strengths and limitations

Strengths

- direct many-to-many MT;
- reduces English pivoting;
- trained explicitly for translation;
- usable without course fine-tuning.

Limitations

- requires huge parallel corpora;
- quality varies by language pair;
- translation-specific, less general-purpose;
- large models are expensive to fine-tune.

Kaggle session

[Open M2M Notebook](#)

- Use pretrained many-to-many model
- No fine-tuning required
- Evaluate multilingual translation quality

M2M-100 hands-on: translation

```
%run m2m_translate.py \  
--model-dir facebook/m2m100_418M \  
--src-lang en \  
--tgt-lang nl \  
--src-file /kaggle/input/datasets/vincentvandeghinste/tatoeba-en-nl/dev.en \  
--out-file test.m2m.nl \  
--ref-file /kaggle/input/datasets/vincentvandeghinste/tatoeba-en-nl/dev.nl \  
--metrics bleu,chrF,ter \  
--batch-size 4 \  
--max-src-len 96 \  
--max-gen-len 96 \  
--num-beams 1
```

Why `%run` instead of `!python`?

In notebook environments, `%run` often gives cleaner output.

- It runs the script inside the current notebook process.
- Progress bars and printed examples can display more neatly.
- It can make hands-on sessions easier to follow.

Metric	Score
BLEU	41.35
chrF	61.59

No training curves are shown because we use the pretrained model directly.

NLLB stands for **No Language Left Behind**.

Its goal is to scale MT to 200+ languages, with special attention to:

- low-resource languages;
- underrepresented language communities;
- data quality and filtering;
- balanced multilingual training.

NLLB: what makes it different?

NLLB is not mainly a new architecture.

Its main innovations are in:

- large-scale parallel data mining;
- aggressive filtering;
- quality control;
- balancing high- and low-resource languages;
- multilingual supervised training at scale.

NLLB language codes

NLLB uses language + script identifiers.

Code	Language
eng_Latn	English, Latin script
nld_Latn	Dutch, Latin script
fra_Latn	French, Latin script
deu_Latn	German, Latin script
ara_Arab	Arabic, Arabic script

Example arguments: `-src-lang eng_Latn` and `-tgt-lang nld_Latn`.

Full list of language codes

[View language codes](#)

Why include the script?

Some languages can be written in multiple scripts.

The script identifier helps the model distinguish:

- language identity;
- writing system;
- expected output script.

This is especially important in large multilingual models.

NLLB: strengths and limitations

Strengths

- very broad language coverage;
- explicit low-resource focus;
- strong supervised MT training;
- high translation quality.

Limitations

- computationally demanding;
- hard to reproduce from scratch;
- depends on mined data quality;
- exact codes are required.

Kaggle session

[Open NLLB Notebook](#)

- Large-scale multilingual MT
- Use language + script codes (eng_Latn)
- Compare with M2M performance

NLLB hands-on: translation

```
%run nllb_translate.py \  
  --model-dir facebook/nllb-200-distilled-600M \  
  --src-lang eng_Latn \  
  --tgt-lang nld_Latn \  
  --src-file /kaggle/input/datasets/vincentvandeghinste/tatoeba-en-nl/dev.en \  
  --out-file dev.nllb.nl \  
  --ref-file /kaggle/input/datasets/vincentvandeghinste/tatoeba-en-nl/dev.nl \  
  --metrics bleu,chrF \  
  --batch-size 32 \  
  --max-src-len 128 \  
  --max-gen-len 128 \  
  --num-beams 4
```

Metric	Score
BLEU	53.00
chrF	70.99

These scores are substantially higher than the M2M-100 scores in this setup.

MADLAD-400 combines two trends:

- large-scale multilingual translation;
- flexible text-to-text modelling.

It occupies an intermediate position between:

- specialised multilingual MT systems such as NLLB;
- general text-to-text systems such as T5.

MADLAD follows a T5-style encoder–decoder design.

It is trained with:

- multilingual parallel translation data;
- large-scale multilingual text corpora;
- text-to-text training objectives.

The result is a multilingual model where translation is central, but the interface remains flexible.

MADLAD-400: translation control

MADLAD can use textual prompts or compact target tags.

Natural-language prompt

Translate English to Dutch: She eats apples.

Target-language tag

<2nl> She eats apples.

The compact tag format is commonly used in practice.

MADLAD language coverage

- Covers **400+** languages
- Trained on large multilingual datasets
- **No explicit published language code list**

Key difference

Unlike mBART or NLLB, MADLAD does **not** rely on a fixed predefined set of language codes.

- Uses flexible tags such as <2n1>
- Language support is **implicit** in training data

MADLAD tags are short target-language indicators.

- <2nl> means: generate Dutch;
- <2fr> means: generate French;
- <2de> means: generate German.

Compared with NLLB, these tags:

- do not explicitly encode script;
- are shorter and easier to type;
- rely more on the model to resolve ambiguity.

MADLAD-400: strengths and limitations

Strengths

- unified text-to-text framework;
- strong multilingual MT quality;
- flexible task specification;
- broad language coverage.

Limitations

- less explicit control than NLLB codes;
- large models need significant resources;
- output can be prompt-sensitive.

Kaggle session

[Open MADLAD Notebook](#)

- Text-to-text multilingual translation
- Use prefix (e.g. <2n1>)
- Compare with NLLB interface

MADLAD hands-on: translation

```
%run madlad_translate.py \  
  --model google/madlad400-3b-mt \  
  --src-file /kaggle/input/datasets/vincentvandeghinste/tatoeba-en-nl/dev.en \  
  --out-file dev.madlad.nl \  
  --ref-file /kaggle/input/datasets/vincentvandeghinste/tatoeba-en-nl/dev.nl \  
  --metrics bleu,chrF \  
  --batch-size 8 \  
  --max-src-len 128 \  
  --max-gen-len 128 \  
  --num-beams 4 \  
  --prefix "<2nl>"
```

Metric	Score
BLEU	56.37
chrF	72.66

MADLAD is competitive with NLLB while offering a more flexible text-to-text interface.

Comparison of pretrained MT architectures

Model	Main training type	Translation control	Main strength
T5	Monolingual span corruption	Text prefix	General text-to-text transfer
mT5	Multilingual span corruption	Text prefix	Multilingual transfer
mBART	Multilingual denoising	Language tags	Seq2seq multilingual MT
M2M-100	Multilingual supervised MT	Language tags	Direct many-to-many MT
NLLB	Large-scale multilingual supervised MT	Language tags	Low-resource coverage at scale
MADLAD-400	Multilingual text-to-text translation	Prompt or target tag	Flexible multilingual translation

Chronological progression

- ① Monolingual denoising or text reconstruction: T5
- ② Multilingual denoising: mT5, mBART
- ③ Massively multilingual supervised translation: M2M-100
- ④ Large-scale multilingual data mining and supervision: NLLB
- ⑤ Multilingual text-to-text translation: MADLAD-400

The trend: translation becomes more central during large-scale training.

A continuum of pretrained MT models

General denoising	→	Multilingual seq2seq transfer	→	Translation-specialised systems
T5, mT5		mBART		M2M-100, NLLB, MADLAD

- Some models must be adapted to MT.
- Some models are already trained as MT systems.
- MADLAD combines a text-to-text interface with translation-specialised training.

What pretrained models mean for MT

Pretrained models make MT more:

Data-efficient

Useful representations are available before fine-tuning.

Multilingual

Shared parameters and tokenisation support cross-lingual transfer.

Reusable

One model can be reused across many language pairs, domains and tasks.

Pretrained MT models and generative LLMs

Encoder–decoder MT models explicitly model:

source text $\rightarrow$ target text

Decoder-only generative LLMs usually treat translation as prompted text continuation.

Key contrast

Pretrained MT models are translation-aligned by architecture or training objective.
Decoder-only LLM translation is often an emergent prompted behaviour.

Key takeaways

- Pretrained MT is not a single method, but a family of approaches.
- Models differ in objective, scale, language coverage and translation control.
- Denoising models need explicit MT fine-tuning.
- M2M-100, NLLB and MADLAD are already translation-specialised.
- The field is moving toward reusable, multilingual, large-scale MT systems.